

Advanced Structures for Priority Queues and Their Extensions

Queue

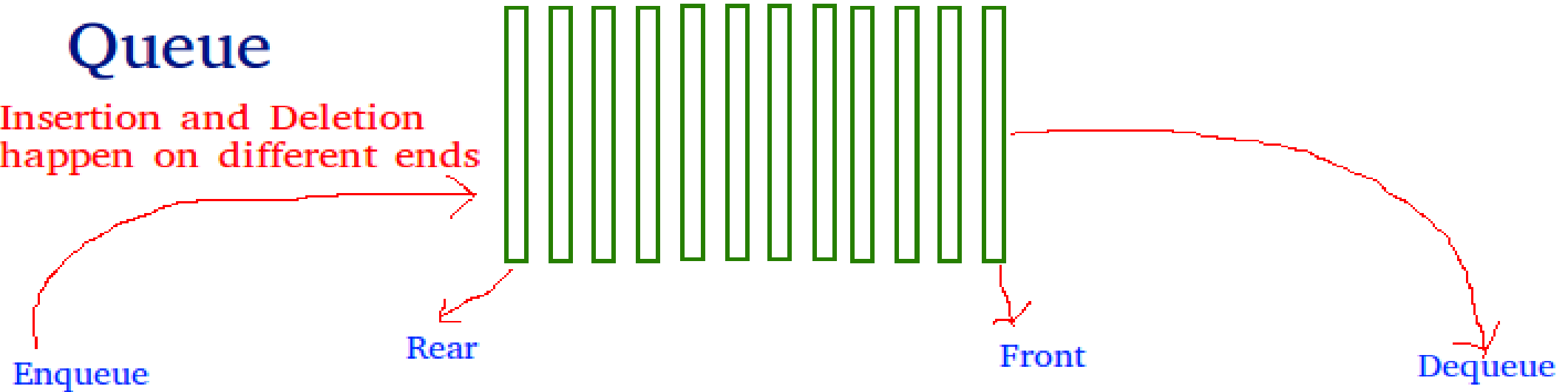
- Like [Stack](#), [Queue](#) is a linear structure
- The order is **F**irst **I**n **F**irst **O**ut (FIFO).
- Example : any queue of consumers for a resource where the consumer that came first is served first.
- The difference between stacks and queues is in removing.
- In a stack we remove the item the most recently added; in a queue, we remove the item the least recently added.

Operations on Queue: Operations on Queue:

- Mainly the following four basic operations are performed on queue:
- **Enqueue:** Adds an item to the queue. If the queue is full, then it is said to be an Overflow condition.
- **Dequeue:** Removes an item from the queue. The items are popped in the same order in which they are pushed. If the queue is empty, then it is said to be an Underflow condition.
- **Front:** Get the front item from queue.
- **Rear:** Get the last item from queue.

Queue

Insertion and Deletion
happen on different ends



First in, first out

Implementation: Linked List

- In a [Queue data structure](#), we maintain two pointers, *front* and *rear*. The *front* points the first item of queue and *rear* points to last item.
- **enQueue()** This operation adds a new node after *rear* and moves *rear* to the next node.
- **deQueue()** This operation removes the front node and moves *front* to the next node.
- **Time Complexity:** Time complexity of both operations enqueue() and dequeue() is $O(1)$ as we only change few pointers in both operations. There is no loop in any of the operations.

Priority Queue

- Priority Queue is an extension of [queue](#) with following properties.
 - Every item has a priority associated with it.
 - An element with high priority is dequeued before an element with low priority.
 - If two elements have the same priority, they are served according to their order in the queue.
- A typical priority queue supports following operations.
 - **insert(item, priority)**: Inserts an item with given priority.
 - **getHighestPriority()**: Returns the highest priority item.
 - **deleteHighestPriority()**: Removes the highest priority item.

How to implement priority queue?

```
struct item {  
    int item;  
    int priority;  
}
```

1. Using Array:

- insert() operation can be implemented by adding an item at end of array in $O(1)$ time.
- getHighestPriority() operation can be implemented by linearly searching the highest priority item in array. This operation takes $O(n)$ time.
- deleteHighestPriority() operation can be implemented by first linearly searching an item, then removing the item by moving all subsequent items one position back.

2. Using Linked List

- Using Linked List, time complexity of all operations remains same as array.
- The advantage with linked list is deleteHighestPriority() can be more efficient as we don't have to move items.

3. Using Heaps:

- Heap is generally preferred for priority queue implementation
- In a Binary Heap, `getHighestPriority()` can be implemented in $O(1)$ time,
- `insert()` can be implemented in $O(\text{Log}n)$ time and
- `deleteHighestPriority()` can also be implemented in $O(\text{Log}n)$ time.

Heap Data Structure

- A Heap is a special Tree-based data structure in which the tree is a complete binary tree. Generally, Heaps can be of two types:
 - **Max-Heap:** In a Max-Heap the key present at the root node must be greatest among the keys present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree.
 - **Min-Heap:** In a Min-Heap the key present at the root node must be minimum among the keys present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree.

Binary Heap

- A Binary Heap is a Binary Tree with following properties.
 1. It's a complete tree. This property of Binary Heap makes them suitable to be stored in an array.
 2. A Binary Heap is either Min Heap or Max Heap. In a Min Binary Heap, the key at root must be minimum among all keys present in Binary Heap. The same property must be recursively true for all nodes in Binary Tree. Max Binary Heap is similar to Min Heap.

How is Binary Heap represented?

- A Binary Heap is a Complete Binary Tree.
- A Binary Heap is typically represented as an array.
- The root element will be at $\text{Arr}[0]$.

$\text{Arr}[(i-1)/2]$

Returns the parent node

$\text{Arr}[(2*i)+1]$

Returns the left child node

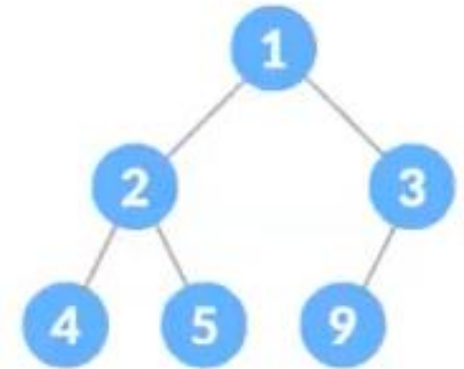
$\text{Arr}[(2*i)+2]$

Returns the right child node

Heap

- A special Tree based data structure
- Tree is a complete binary tree
- Also called as binary heap
- Satisfy Heap Property
- Two Types
 - Min Heap : key of each node is always smaller than the child node/s and the key of the root node is the smallest among all other nodes.
 - Max Heap : key of each node is always greater than its child node/s and the key of the root node is the largest among all other nodes;

Min Heap



Max Heap



Applications of Heaps:

- Heap Sort: Heap Sort uses Binary Heap to sort an array in $O(n \log n)$ time.
- Priority Queue: Priority queues can be efficiently implemented using Binary Heap
- Graph Algorithms: The priority queues are especially used in Graph Algorithms like Dijkstra's Shortest Path and Prim's Minimum Spanning Tree.

Representation

- A Binary Heap is a Complete Binary Tree.
- A binary heap is typically represented as an array.
- The root element will be at Arr[0].
- Below table shows indexes of other nodes for the ith node, i.e., Arr[i]:

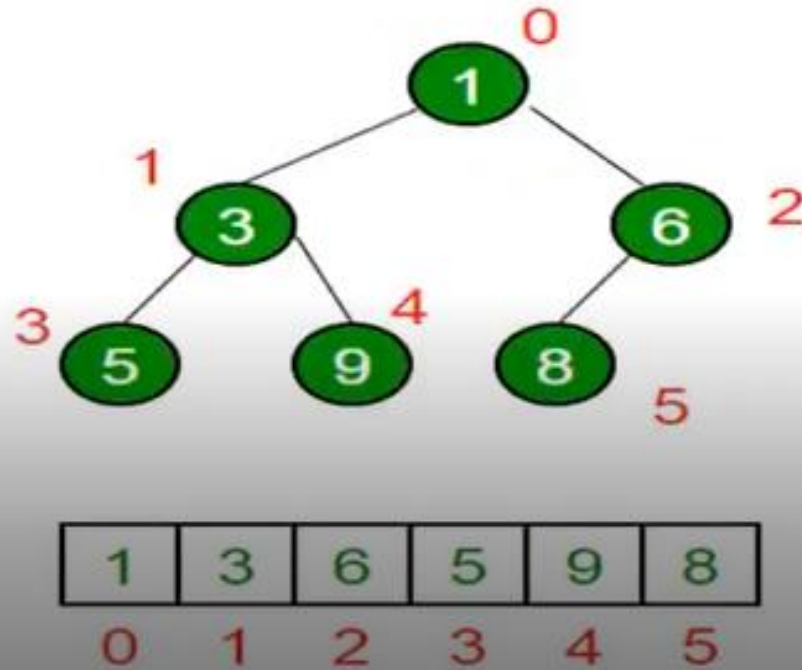
$\text{Arr}[(i-1)/2]$ Returns the parent node

$\text{Arr}[(2*i)+1]$ Returns the left child node

$\text{Arr}[(2*i)+2]$ Returns the right child node

Traversal

- The traversal method use to achieve Array representation is **Level Order**



Operations

- **Heapify** : Heapify is the process of creating a heap data structure from a binary tree. It is used to create a Min-Heap or a Max-Heap.
- **Peek (Find Max/ Min)** : Peek operation returns the maximum element from Max Heap or minimum element from Min Heap without deleting the node. For both Max heap and Min Heap. Time Complexity of this operation is $O(1)$.
- **Extract-Max/Min** : Extract-Max returns the node with maximum value after removing it from a Max Heap whereas Extract-Min returns the node with minimum after removing it from Min Heap. Time Complexity of this Operation is $O(\log n)$ as this operation needs to maintain the heap property (by calling heapify()) after removing root.

- **insert():** Inserting a new key takes $O(\log n)$ time. We add a new key at the end of the tree. If new key is greater than its parent, then we don't need to do anything. Otherwise, we need to traverse up to fix the violated heap property.
- **delete():** Deleting a key also takes $O(\log n)$ time. We replace the key to be deleted with minimum infinite by calling decreaseKey(). After decreaseKey(), the minus infinite value must reach root, so we call extractMin() to remove the key.

Heap Implementation

Initialization

```
void initHeap(Heap* h, int size) {  
    h->A = (int*)malloc(size * sizeof(int));  
    h->size = size;  
    h->rear = -1;  
}
```

Swap Function

```
void swap(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

Heap Implementation

Queue Empty

```
int isEmpty(Heap* h) {  
    return (h->rear == -1);  
}
```

Queue Full

```
int isFull(Heap* h) {  
    return (h->rear == h->size - 1);  
}
```

Heap Implementation

Heap Sort Function

```
void HeapSort(int arr[], int size) {  
    Heap newHeap;  
    initHeap(&newHeap, size);  
  
    for (int i = 0; i < size; i++) {  
        insert(&newHeap, arr[i]);  
    }  
  
    int i = 0;  
    while (!isEmpty(&newHeap)) {  
        arr[i] = Remove(&newHeap);  
        i++;  
    }  
}
```

Heap Implementation

Insert Function

```
void insert(Heap* h, int value) {  
    if (h->rear == h->size - 1) {  
        printf("Heap is full. Cannot insert value.\n");  
        return;  
    }  
  
    h->rear++;  
    int i = h->rear;  
  
    while (i > 0 && value > h->A[(i - 1) / 2]) {  
        h->A[i] = h->A[(i - 1) / 2];  
        i = (i - 1) / 2;  
    }  
  
    h->A[i] = value;  
}
```

Heap Implementation

Remove Function

```
int Remove(Heap* h) {  
    if (h->rear < 0) {  
        printf("Heap is empty. Cannot remove value.\n");  
        return -1;  
    }  
  
    int removedValue = h->A[0];  
    h->A[0] = h->A[h->rear];  
    h->rear--;  
    heapify(h, 0);  
  
    return removedValue;  
}
```

Heap Implementation

Heapify Function

```
void heapify(Heap* h, int index) {
    int largest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left <= h->rear && h->A[left] > h->A[largest])
        largest = left;

    if (right <= h->rear && h->A[right] > h->A[largest])
        largest = right;

    if (largest != index) {
        swap(&h->A[index], &h->A[largest]);
        heapify(h, largest);
    }
}
```